COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/22/26	**DATE SUBMITTED:**	9/22/26
NAME:	Reyes, Cyrill Joferson B.	**INSTRUCTOR:**	Engr. Jimlorn M. Quejado

ACTIVITY NO. 9.1 Tree ADT

1. PROCEDURE OUTPUT

Act9.1 > main.cpp > ...

```cpp
#include <iostream>
#include "BST.h"


int main(){
    //create the root node:
    BinarySearchTree<int> tree(20);

    tree.insert(10);
    tree.insert(5);
    tree.insert(70);
    tree.insert(80);
    tree.insert(40);
    tree.insert(50);

    std::cout<<"preOrder: ";
    tree.preOrder();
    std::cout<<std::endl;


    std::cout<<"inOrder: ";
    tree.inOrder();
    std::cout<<std::endl;

    std::cout<<"postOrder: ";
    tree.postOrder();
    std::cout<<std::endl;


    return 0;
}
```

Act9.1 > C BST.h > ...

```h
#ifndef BST_H
#define BST_H

#include <iostream>

template <typename T>
class BinarySearchTree {
private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

public:
    // Constructor
    BinarySearchTree(T value);

    // Member functions
    void insert(T value);

    // Traversing algorithms
    void preOrder();
    void inOrder();
    void postOrder();
};

// Constructor implementation
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value) {
    key = value;
    left = nullptr;
    right = nullptr;
}

```

Act9.1 > C BST.h > ...

```
1   #ifndef BST_H
34  // Insert implementation
35  template <typename T>
36  void BinarySearchTree<T>::insert(T value) {
37      if (value < key) {
38          if (left == nullptr) {
39              left = new BinarySearchTree<T>(value);
40          } else {
41              left->insert(value);
42          }
43      } else if (value > key) {
44          if (right == nullptr) {
45              right = new BinarySearchTree<T>(value);
46          } else {
47              right->insert(value);
48          }
49      }
50  }
51
52  // PreOrder traversal implementation
53  template <typename T>
54  void BinarySearchTree<T>::preOrder() {
55      std::cout << key << " ";
56      if (left != nullptr) {
57          left->preOrder();
58      }
59      if (right != nullptr) {
60          right->preOrder();
61      }
62  }
63
```

```
63
64  // InOrder traversal implementation
65  template <typename T>
66  void BinarySearchTree<T>::inOrder() {
67      if (left != nullptr) {
68          left->inOrder();
69      }
70      std::cout << key << " ";
71      if (right != nullptr) {
72          right->inOrder();
73      }
74  }
75
76  // PostOrder traversal implementation
77  template <typename T>
78  void BinarySearchTree<T>::postOrder() {
79      if (left != nullptr) {
80          left->postOrder();
81      }
82      if (right != nullptr) {
83          right->postOrder();
84      }
85      std::cout << key << " ";
86  }
87
88  #endif
```

```
PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes> cd "c:\Us
$?) { g++ main.cpp -o main } ; if ($?) { .\main }
preOrder: 20 10 5 70 40 50 80
inOrder: 5 10 20 40 50 70 80
postOrder: 5 10 50 40 80 70 20
PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes\Act9.1>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

BST.h Act9.1 | main.cpp Act9.1 | BST.h ...\supplementa

Act9.1 > supplementary > main.cpp > main()

```cpp
#include <iostream>
#include "BST.h"

using namespace std;

int main() {
    Node* tree = nullptr;
    int items[] = {2, 3, 9, 18, 0, 1, 4, 5};

    for (int n : items) {
        tree = insert(tree, n);
    }

    cout << "In-order: ";
    inOrder(tree);
    cout << "\n";

    cout << "Pre-order: ";
    preOrder(tree);
    cout << "\n";

    cout << "Post-order: ";
    postOrder(tree);
    cout << "\n";

    return 0;
}
```

Act9.1 > supplementary > C BST.h > inOrder(Node *)

```cpp
#pragma once
#include <iostream>

struct Node {
    int val;
    Node* left;
    Node* right;

    Node(int v) : val(v), left(nullptr), right(nullptr) {}
};

inline Node* insert(Node* root, int val) {
    if (!root) return new Node(val);

    if (val < root->val) {
        root->left = insert(root->left, val);
    } else if (val > root->val) {
        root->right = insert(root->right, val);
    }
    return root;
}

inline void inOrder(Node* root) {
    if (!root) return;
    inOrder(root->left);
    std::cout << root->val << " ";
    inOrder(root->right);
}

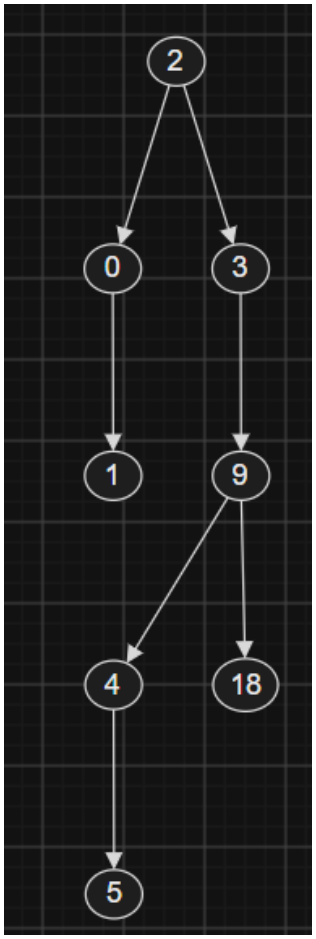
inline void preOrder(Node* root) {
    if (!root) return;
    std::cout << root->val << " ";
    preOrder(root->left);
    preOrder(root->right);
}

```

```
37  inline void postOrder(Node* root) {
38      if (!root) return;
39      postOrder(root->left);
40      postOrder(root->right);
41      std::cout << root->val << " ";
42  }
```

```
PS C:\Users\Jofjof\OneDrive\D
ntary\" ; if ($?) { g++ main.
In-order: 0 1 2 3 4 5 9 18
Pre-order: 2 0 1 3 9 4 5 18
Post-order: 1 0 5 4 18 9 3 2
PS C:\Users\Jofjof\OneDrive\D
```

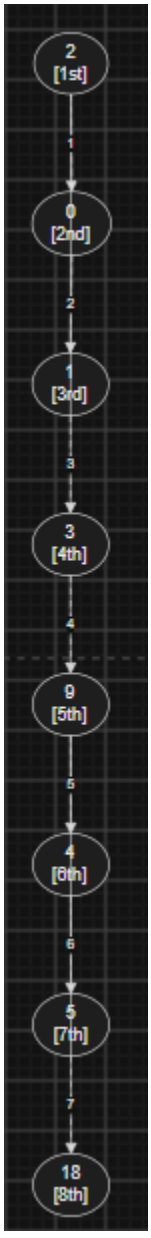
Tree Diagram



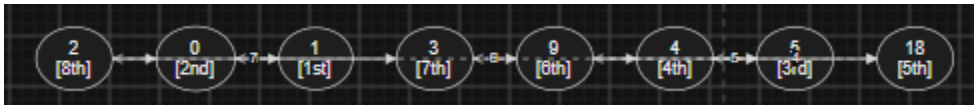
In-Order Traversal



Pre-Order Traversal



Post-Order Traversal



COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code formatting, syntax checking, and grammar review**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was strictly used as a proofreading and formatting tool to verify C++ syntax and improve writing clarity. All problem-solving, tree traversals, and main logic were completed independently**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.